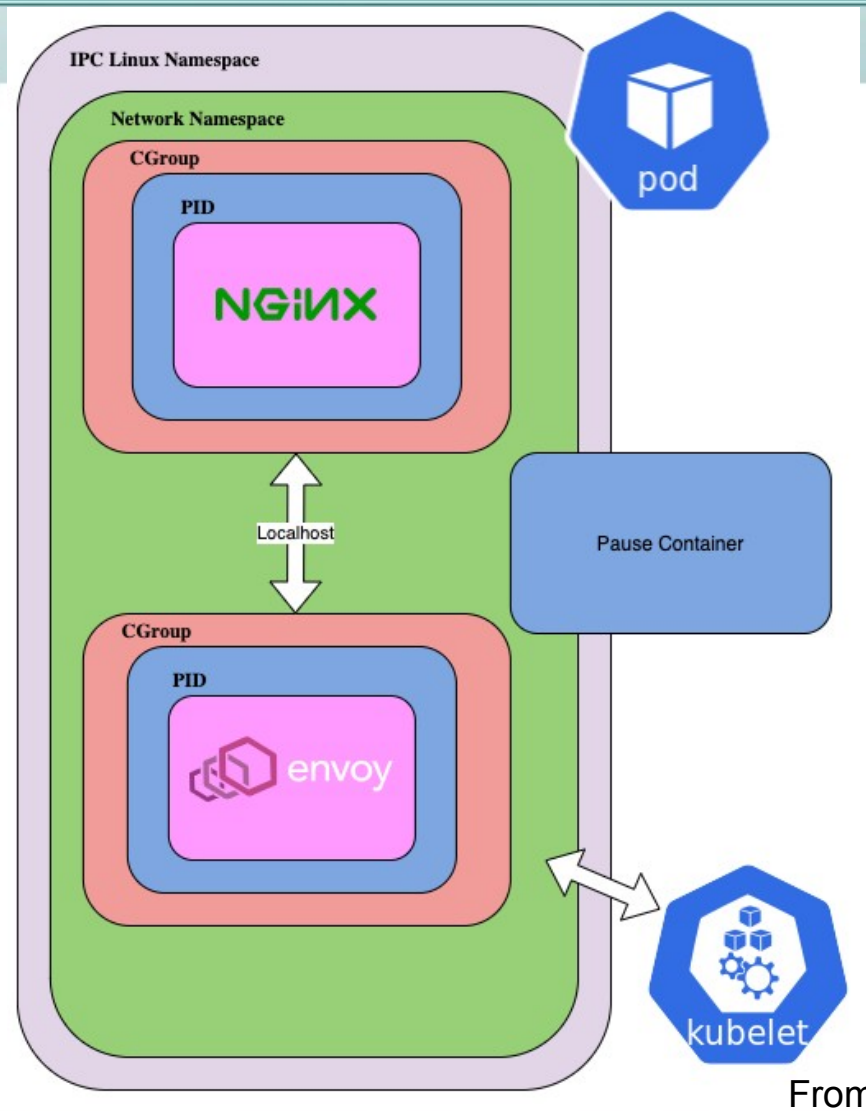
2. Why the Pod?

- What is a Pod, high level
- Example web application
- What is a Pod, more detail
- Example: create a web application
- Scaling, HA and the Control Plane
- Summary

Pods

- A pod is a colocated group of containers that share an IP address as well as host node
  - Linux implementation uses namespaces and cgroups, other implementations possible
- Node allows containers in a pod to share files and configuration information
- Pod typically deployed using Deployment
- Pod is smallest atomic unit deployed to K8S

Pod – Better than a VM



- Pod is
  - One or more containers
  - Sharing one IP on a host
  - Containers share localhost
- Container is
  - Encapsulated environment
    - own Linux namespaces and control group
  - Run by container engine
  - Runs a container image
    - build of image layers
    - can be stored in registry

From Core Kubernetes MEAP, by Jay Vyas and Chris Love, 2021

Zeus' Zap Web Example

- Energy drink company with 3-tier web app
  - UI, Middle layer, Database
- Front end is in JavaScript served by Nginx
- Middle is two web-controller python microservices hosted by django
- Backend is CockroachDB serving on port 6379, backed by persistent storage
- Each component runs in a container

Starting the App with Docker

- Suppose production system started with docker run:

```
$ docker run -t -i ui -p 80:80
$ docker run -t -i microservice-a -p 8080:8080
$ docker run -t -i microservice-b -b 8081:8081
$ docker run -t -i cockroach:cockroach -p 6379:6379
```

- Problems with this:
  - need load balancing in front of port 80
  - moving db container requires letting other containers know new DB's IP
  - if DB instance dies, need to get its data to a new instance

Other Problems with Container-only

- May be conflict with other processes binding to port 80
  - need service-aware network load balancing
  - load balancing should also follow containers if they move
- If containers can move between hosts, need to migrate storage volumes
  - need storage-aware scheduling to keep processes with data
- Running multiple containers on each host may lead to contention for resources (memory, CPU)
- Solving problems needs setting up some infrastructure

Requirements for Infrastructure

- Need the following to run the application
  - Virtual machines or physical servers
  - load balancing
  - application discovery
  - storage
  - security
  - scheduling based on CPU and memory requirements
  - logging, monitoring, continuous integration/continuous delivery, backups, secrets management

Additional Operational Issues: not Covered by Containers

- Zeus' Zap sponsors e-sport events that drive up traffic during contests (burst traffic)
- Costs for downtime is huge
- Nice to be able to scale down as well to save costs
- Adds these requirements:
  - scaling
  - high availability
  - versioning apps to allow rollbacks
  - cost management (especially if using public clouds)

What is a Pod?

- One of more OCI images, running as containers, on a node of a Kubernetes cluster
  - Recall a node is a server that runs `kubelet`
  - Both nodes and pods are API objects
- Can deploy pod thusly:

```
cat << EOF > pod.yaml
apiVersion: v1
kind: Pod
metadata:
spec:
  container:
    - name: busybox
      image: myregistry.io/foo
EOF

kubectl create -f pod.yaml
```

- But normally use
  - Deployments
  - Jobs
  - StatefulSets
  - DaemonSets

What are Deployments, etc. ?

- Deployments are API object that specifies the characteristics of pods to deploy, with related services
  - create ReplicaSets which create the Pods (for scaling)
- Jobs are used to run a pod as a batch process
- StatefulSets host applications with persistent state
  - ordinal pod naming for unique network identifiers, persistent storage mounted to same pod, ordered starting, scaling and updates of pods
- DaemonSets run a pod on every node of a cluster
  - for things such as networking, storage, logging

Linux Mechanism - Namespaces

- Linux namespaces are not Kubernetes namespaces
- Linux namespaces restrict the resources a process (and its children) can *see* on the host machine
- On process creation, kernel can give processes a namespace with, for example:
  - hostname, network, mounts, user ids, process ids
- Namespaces are controlled with flags
  - E.g. CLONE_NEWUTS, CLONE_NEWPID, CLONE_NEWNL
  - See `man namespaces`

A Pod is a bunch of Namespaces

- These Linux namespaces are in a Pod:
  - one or more PID namespaces
  - single networking namespace
  - IPC namespace
  - cgroup (Control Group) namespace
  - mnt (mount) namespace
  - user (User id) namespace
- Allows pod isolation from other processes on the host
  - virtual networking connects to software defined network (SDN) for the Kubernetes cluster

Kubernetes Node

- Node is server with specified components
  - installed OS
  - systemd - service manager
  - Container Runtime Interface (CRI) – starts/stops containers
  - Container Networking Interface (CNI) - network for pods
  - kubelet - node agent for containers and pod lifecycle
  - kube-proxy – front end for Kubernetes services
    - handles load balancing for pods behind a service
- Lots of Pods

Node Operating System

- Linux most common
  - MS Windows Server 2019 also possibility
- Common to use OS distribution tailored for container clusters
- OKD uses open-source Fedora CoreOS (FCOS) or Stream Centos CoreOS (SCOS)
  - see https://getfedora.org/en/coreos?stream=stable
  - Current size ~760MB (ISO)

Fedora CoreOS (FCOS)

- Descendent of CoreOS Container Linux and Fedora Atomic Host
- Automatically updating, minimal OS for running containerized workloads securely and at scale
- Optimized for Kubernetes but also used stand alone
- Uses ignition tool for provisioning
- Uses rpm-ostree to install updates
- Hardened with SELinux
- OKD updates also update FCOS versions

Node: Kubelet + CNI + CRI

- Kubelet is binary program running on every node
- Communicates with API server
- Ensures
  - any pod scheduled on the node is running
  - API server is aware that kubelet is healthy
  - unused storage and network devices are garbage collected
- Uses CRI and CNI to keep Pod state consistent with control plane
  - CRI interfaces with container engine, such as cri-o

Node: Services and kube-proxy

- Services proxy multiple Pods of the same type
- Every node needs to be aware of every service and every pod.
- kube-proxy manages each service on each node
- To do this, kube-proxy uses
  - ClusterIP to load-balance Pods for internal visibility
  - NodePort to expose services to external IPs
  - LoadBalancer for external load balancers

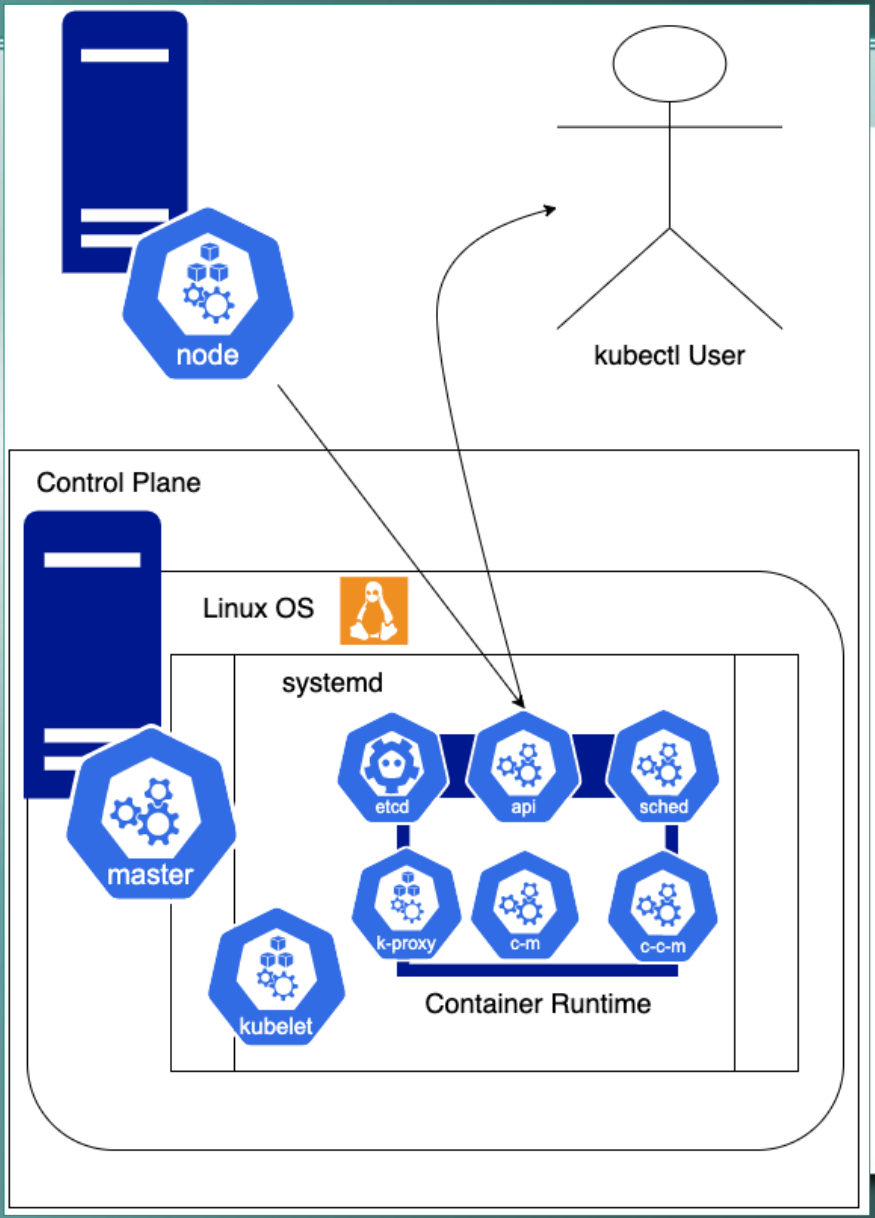
Examining Nodes

- Basic command: `kubectl get no`
  - find names of nodes
- More details for given named node:
  - `kubectl get no` *`name-of-node`* `-o yaml`
- See sections
  - status: information about the node
  - images: images running on the node
  - nodeInfo: versions of main systems running on node

Control Plane: Keeps it Together

- Covers API server, scheduler and controller manager
- Connects to etcd (Kubernetes database)
- Control plane components run on nodes
  - called control plane nodes (or sometimes master nodes)
  - 3 control plane nodes on OKD for fault tolerance
- Controller manager interacts with vendor technologies
  - recently refactored to cloud-controller-manager
- Control plane allows fault tolerance, scalability, as well as cost savings (containers and serverless)

Control Plane Node



- API Server
  - kube-apiserver
- Scheduler
  - kube-scheduler
- Controller Manager
  - kube-controller-manager
  - cloud-controller-manager
- kubelet
  - basic node component
  - ensures pods are running

From Core Kubernetes MEAP, by Jay Vyas and Chris Love, 2021

Creating Pod with kubelet

- How would something like `docker run -v` *`tmp`*`:ctemp -p 8080:8080 nginx` be in K8S?
  - create PodSandbox with networking for pod
  - create hostPath volume and mount into pod
  - set up pod networking via CNI provider
  - start container engine to create pause container for cgroup parents of Nginx container
  - start container engine to mount volume into pod and start the image

kubelet uses CRI Interfaces

- Recall pod is group of containers in isolated environment with resource constraints
  - can be implemented using virtual machines or Linux namespaces and cgroups
- CRI PodSandboxManager creates environment for pods
- CRI ContainerRuntime starts, stops, executes containers
- CRI ImageService pulls, lists, removes images

Create Web App with kubectl

- Suppose run kubectl create -f deployment.yaml
- kubectl communicates to the kube-apiserver server in the control plane
- kube-scheduler assigns pod to node
- Operators can be used to manage stateful requirements for applications
- Operators usually define custom resource definitions to create new API objects
- More details on following slides

Deployments Create Pods

- In Kubernetes, users don’t create pods manually
- Most pods created with Deployments
  - Which create ReplicaSets (under the covers) to allow scaling up and down
- Pods can be scheduled on nodes based on labels in descriptor (not discussed further here)
- Background tasks can be run on nodes automatically with DaemonSets (usually part of infrastructure)

Example Deployment Definition

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-openshift
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello-openshift
  template:
    metadata:
      labels:
        app: hello-openshift
    spec:
      containers:
      - name: hello-openshift
        image: openshift/hello:latest
        ports:
        - containerPort: 80
```

Suppose this is in hello.yaml file

To deploy this:
oc create -f hello.yaml

Will create pod in your current project

Can also use console!

use oc get deployments to list, oc describe deployment xxx to see YAML description

kube-apiserver

- kube-apiserver is an HTTP-based (stateless) REST server exposing the API objects
  - usually runs on all control plane nodes
  - load balancer will send the request to one of them
  - uses etcd to store API objects - and CRUD ops on them
  - communication is authenticated and authorized
  - allows web-hook callbacks to integrate with other systems
  - components use API server to update their status
  - control plane watches API server for changes in objects

Kubernetes Scheduler

- Considers multiple factors in Pod scheduling
  - hardware components on Nodes, free space, scheduling constraints, and weighting factors
- Allows affinity and anti-affinity rules to attract or repel Pods from Nodes
- *Taints* allow a Node to repel a set of Pods so scheduler can avoid them
- Previous example schedules 3 replicas for fault tolerance and scaling
- Once Node is determined, kubelet deploys it

Infrastructure Controllers: Example

- Zeus' Zap application uses CockroachDB

```
$ kubectl apply -f https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/config/crd/bases/crdb.cockroachlabs.com_crdbclusters.yaml
$ kubectl apply -f https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/manifests/operator.yaml
```

- These CRDs install the CockroachDB operator
- Then download example

```
$ curl -LO https://raw.githubusercontent.com/cockroachdb/cockroach-operator/master/examples/example.yaml
```

- see next slide

example.yaml to create DB

```
apiVersion: crdb.cockroachlabs.com/v1alpha1
kind: CrdbCluster
metadata:
  name: cockroachdb
spec:
  dataStore:
    pvc:
      spec:
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: "60Gi"
        volumeMode: Filesystem
  resources:
    requests:
      cpu: "2"
      memory: "8Gi"
```

```
    limits:
      cpu: "2"
      memory: "8Gi"
  tlsEnabled: true
  image:
    name:
cockroachdb/cockroach:v21.1.5
  nodes: 3
  additionalLabels:
    crdb: is-cool
```

- has operator create:
  - TLS Keys
  - StatefulSet for DB with PV/PVC (storage)
  - services
  - resource budget

Infrastructure Controllers at Work

- Kubernetes Controller Manager (k-c-m) and Kubernetes Cloud Controller Manager (c-c-m)
- For previous example need storage for the StatefulSet
  - defined by Persistent Volumes (PV) and Persistent Volume Claims (PVC) that create storage definitions
- k-c-m is set of control loops on a control plane node
- k-c-m detects storage is required and creates the storage and mounts it on the Node
- kubelet then attaches storage to container using mnt Linux namespace

Why k-c-m and c-c-m?

- These controllers need to interface with public cloud provider services
  - when K8S is running on an infrastructure cloud
- k-c-m is one giant code base with interfaces to all the different clouds (AWS, Azure, GCP, etc)
- It is being refactored into c-c-m to use interfaces that the cloud providers can implement
  - outside of the Kubernetes code base
- Note the pattern: design to interfaces!

Infrastructure Controllers

- k-c-m/c-c-m provide following controllers:
  - Node controller – watch for Pod health and scaling
  - Route controller – set up routes in underlying cloud infrastructure
  - Service controller – create, update, delete cloud provider load balancers
  - Volume controller – create, attach and mount volumes

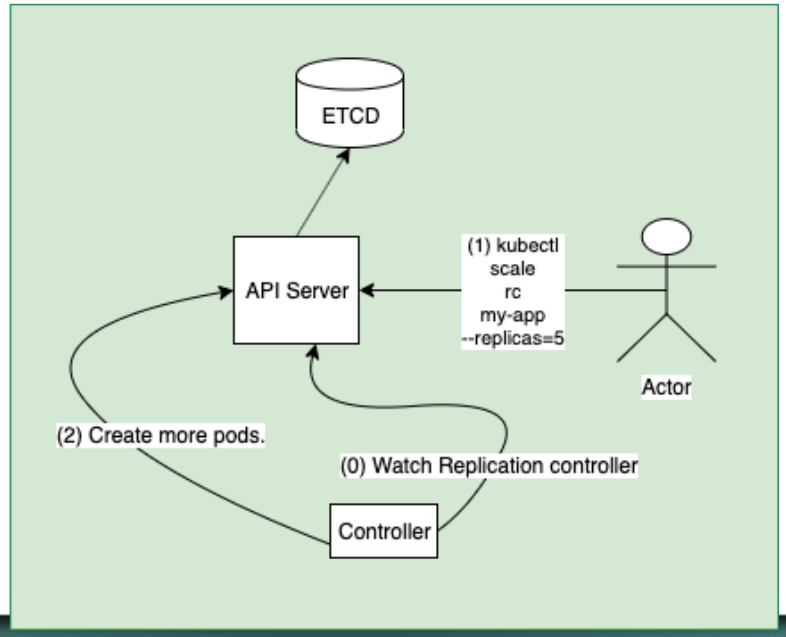
Zeus' Zap Needs LoadBalancer Service

- Specify LoadBalancer in yaml file for service
- k-c-m notices change and creates load balancer
  - makes API calls to create or use external load balancer
- kube-proxy then routes traffic
  - first to Node then to Pods

```yaml
apiVersion: v1
kind: Service
metadata:
  name: example-service
spec:
  selector:
    app: example
  ports:
    - port: 8765
      targetPort: 9376
  type: LoadBalancer
```

Scaling

- kubectl scale can increase or decrease number of Pods in ReplicaSet for Deployment or StatefulSet
  - DaemonSets and Jobs do not have ReplicaSets
- E.g. `kubectl scale --replicas 300 deploy` *`name`*



Outages?

- Pod fails
  - kubelet attempts to restart, repeatedly with backoffs
- Node fails
  - kubelet stops updating API that node is healthy
  - node status changed to off-line: pods not scheduled there
  - Pods on node scheduled for deletion and rescheduled to other nodes
- Software updates
  - can do rolling update that updates one Pod at a time, service always available

Section Summary

- Pod is basic Kubernetes API object
- Pods deployed with Deployment, StatefulSet, Job, DaemonSet
- Controllers create and manage Pod lifecycles
- Control plane makes it all work
  - bind storage to processes, create and scale containers, kill and migrate unhealthy containers, create IP routes to ports, update load-balanced endpoints, etc.
- kube-apiserver provides RESTful services to access all services for Kubernetes